Points of significance

https://doi.org/10.1038/s41592-023-01973-1

Convolutional neural networks

By Alexander Derry, Martin Krzywinski & Naomi Altman

Check for updates

Nature uses only the longest threads to weave her patterns, so that each small piece of her fabric reveals the organization of the entire tapestry.
—Richard Feynman

In the previous column, we introduced the basic building blocks of an artificial neural network (ANN): neurons computing linear combinations of inputs, followed by a nonlinear activation function[1]. In our ANN example, each neuron was connected to all other neurons in neighboring layers, making it a 'fully connected' network (FCN). FCNs are very effective for tabular data in which rows and columns do not have an inherent order (for example, gene expression). However, when spatial (for example, sequences, images) or temporal (for example, electrocardiograms) order is critical to interpretation, FCNs are less suitable. This month, we will explore convolutional neural networks (CNNs), which overcome this limitation.

Consider the task of using a protein's sequence to predict whether it localizes to the nucleus. Let's see how we might solve this problem with an FCN with a single hidden layer.

First, we need to create a numerical representation of a sequence. Since each sequence position can take on one of 20 values (amino acids), we can encode it with a so-called 'one-hot' vector consisting of 20 elements that are all zero except for a single nonzero (for example, 1) element that uniquely identifies the amino acid (Fig. 1a). For example, alanine might be [1, 0, 0, ...], cysteine [0, 1, 0, ...] and so on. To encode a sequence with, for example, $N = 16$ amino acids, we'll need 16 such vectors with a total of $20 \times 16 = 320$ elements, which we can more generally call 'features'.

To create an FCN, we connect each neuron in the hidden layer to each input feature, which requires 321 learnable weights W_{ij} (including the bias b) per neuron. If we have H hidden neurons and a single binary output neuron, the model would have a total of $322H + 1$ parameters (Fig. 1a).

Even for short sequences, the number of parameters grows quickly. For $H = 160$ (half of the number of features, which is an arbitrary but reasonable choice), the FCN would need to learn a total of $321 \times 160 + 161 = 51{,}512$ parameters. If we wanted to analyze proteins of more typical size (for example, $N = 300$), we would be dealing with 6,000 features and (at least) 960,321 parameters – likely more, since additional (and larger) hidden layers would be needed to capture complex hierarchical patterns. Such a complex model would not only be highly susceptible to overfitting[2] (unless the sample size is very large), it would be computationally expensive to train.

Luckily, there is a way to substantially reduce the number of parameters and even improve performance. Specifically, if we assume that the protein's localization is due to short patterns, we can rearrange the network into a different 'architecture' to leverage the spatial dependence created by the inherent order of the amino acids (residues). This is something our basic FCN is not sensitive to: shuffling the input sequence has no impact.

We do this by first replacing the hidden layer, which contains many neurons that each process all residues, with a convolutional layer that has a relatively small number of neurons that process residues only within a small window (Fig. 1b). These neurons are commonly called 'filters' or 'kernels' because they are equivalent to convolution filters or kernels in image processing (for example, smoothing). CNNs usually have multiple filters; each is randomly initialized and converges to a different set of parameters during training[1], enabling it to detect a different pattern.

In our example (Fig. 1b), the convolution layer has 3 filters, each processing a window of $F = 3$ (filter size) residues at a time (centered on a given sequence position). Each filter has $20F + 1 = 61$ parameters and, crucially, these are shared among all positions (and updated as the window steps along the sequence), which dramatically reduces the size of the model. Each filter outputs a unique activation value

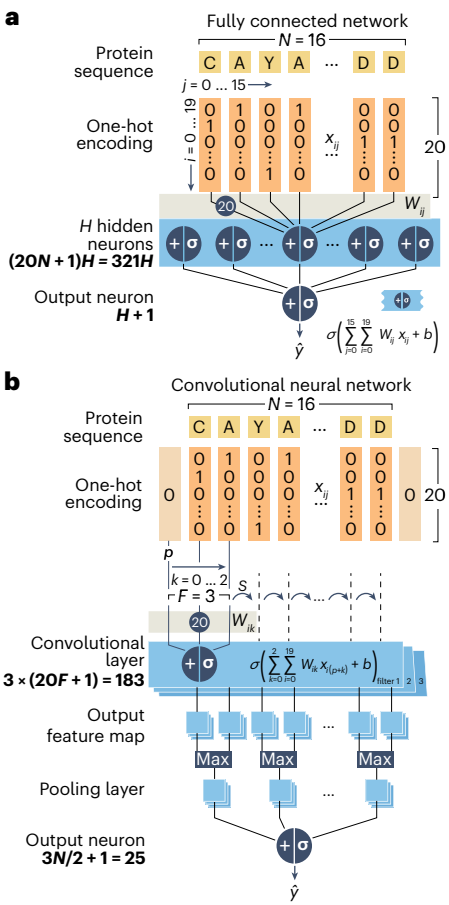


Fig. 1 | Comparison between fully connected networks and convolutional neural networks. Input is a length $N = 16$ protein sequence in which the jth amino acid is encoded by a one-hot column vector of length 20 (orange). i, j, k and p are zero-indexed. **a**, In an FCN, each x_{ij} is connected to each of H hidden neurons (blue), which compute linear combination (+) and activation (σ), by way of 321 weights W_{ij} (including one bias, not shown). This FCN has $322H + 1$ parameters (sum of bold numbers). **b**, A CNN replaces the hidden layer with convolutional and pooling layers (blue). The convolutional layer has one or more filters (for example, 3, shown as a stack), which calculate a unique activation value for each position over a sliding window of size F (for example, 3). The window steps along the sequence with a stride $S = 1$ (black curved arrow) during training. For each filter window starting position p, the filter's weights W_{ik} are multiplied by a window $x_{i(p+k)}$ to generate an output feature map, which is then max-pooled (for example, in pairs) and passed to a single fully connected layer with a sigmoid activation function. A filter's $20F + 1 = 61$ parameters (weights W_{ik} and bias b) are shared among all positions. This CNN has $183 + 25 = 208$ parameters. Input data are padded (light orange) to accommodate the sliding window at edge positions. Only connections through the first filter are shown; the other two filters have their own weights and are wired analogously.

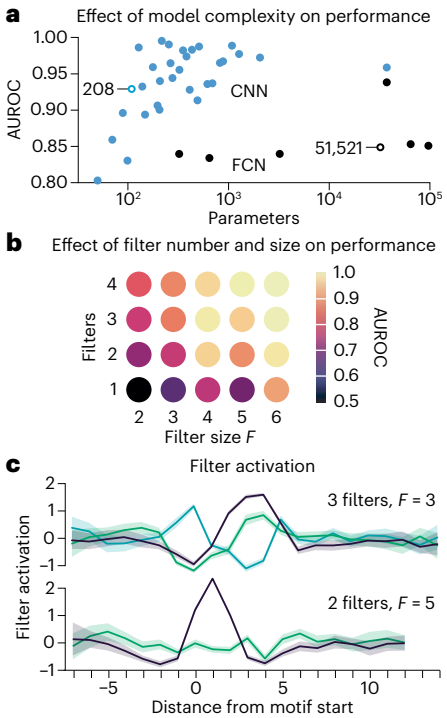


Fig. 2 | Impact of hyperparameter selection. a, The effect of the number of parameters on the area under the receiver operating characteristic curve (AUROC, averaged over five random initializations) for various CNNs (blue, different numbers of filters and filter sizes) and FCNs (black, different hidden layer sizes 1–300) when applied to the hold-out set. **b**, Effect of number of filters and filter size on CNN AUROC, shown as an average over five random training initializations. **c**, Average activations of each filter (different colors) at each position in the sequence relative to the start of the nuclear localization motif, computed over all localizing sequences in the test set.

for each position, which, taken together, creates an output feature map. Since our convolutional layer has 3 filters, the size of this map is $3N = 48$ (Fig. 1b).

To further reduce the network's complexity and make it invariant to small translations of the input patterns, each pair of values of a filter's output feature map is 'max-pooled' by replacing them by their maximum. Other pooling operations are possible (for example, mean-pooling), but max-pooling is common because it can capture local features without considering background variation.

Finally, the pooling layer feeds into a single fully connected neuron with a sigmoid activation function, which computes the final output probability. Because both FCNs and CNNs are based on neurons with the same functional form, this fully connected layer is mathematically equivalent to a convolutional layer in which the filter size is equal to the input size ($F = N$).

This CNN has a total of 208 parameters: $3 \times 61 = 183$ in the convolution layer and $3 \times 16/2 + 1 = 25$ in its output neuron. The filter number and filter size are so-called 'hyperparameters' because they correspond to a fixed (not learned) parameter that is a designed feature of the model (just like H in an FCN).

To concretely demonstrate the difference in FCN and CNN performance, let's build a synthetic dataset of protein sequences with $N = 16$ amino acids. Suppose that a sequence localizes to the nucleus if it contains the following motif: two positively charged amino acids (K, H or R), followed by any amino acid, followed by two negatively charged amino acids (D or E). We'll randomly generate 500 localizing sequences, each with a randomly sampled instance of the motif (for example, RHCED) and 500 non-localizing sequences, each with a randomly sampled instance of only the upstream (for example, RH) or downstream (ED) pair. For each class, we'll use 400 of the sequences for training and reserve 100 as a hold-out set to evaluate our trained models.

We'll use the FCN and CNN described above, with 51,521 and 208 parameters, respectively, and measure performance with the area under the receiver operating characteristic curve (AUROC[3]). Even though the CNN has fewer parameters than even a single hidden neuron in the FCN, it outperforms the FCN by about 9% (AUROC 0.929 versus 0.849) (Fig. 2a).

We can improve our CNN's performance even further by exploring different numbers of filters and filter sizes (Fig. 2b). With only one filter, the CNN performs poorly, unless we use a larger filter of size 6 – to be expected since the localizing motif length is 5 amino acids. We can mitigate the low performance of individual short filters by using more than one: 3 filters of size 3 perform similarly to 2 filters of size 5. And using 3 filters of size 6 we can achieve almost perfect classification (AUROC > 0.98).

By plotting the activations of each filter at each position in the input sequence, we can develop an intuition of how the filters respond (Fig. 2c). Filters of size 3 cannot capture the full motif; instead, each filter responds to different (and short) aspect of the motif and their outputs are combined to make the final classification (Fig. 2c, top). However, a filter of size 5 produces high activation across the whole motif (Fig. 2c, bottom).

The basic building blocks in our CNN example could be arranged in more complex ways. For example, a CNN with one convolutional layer is limited in how it combines the outputs of multiple filters, but a two-layer network may perform better even with small filters because of its ability to detect second-level patterns between the filter activations. A CNN may also have filters of various sizes, alternating convolutional and pooling layers, and multiple fully connected layers to compute the final output.

A key strength of CNNs is the ability of each filter to detect the pattern regardless of where in the sequence it occurs: a shift in the location of the input pattern results in an equal shift in the location of the output features, a property known as 'translation equivariance'. CNNs can be easily extended to operate on 2D (image) and 3D (video) inputs, by changing the size and shape (for example, 3 × 3 square) of the convolutional filters, which would step along multiple dimensions of the data.

However, CNNs are not without limitations. For example, they do not explicitly model the sequential nature of the input data, so subtle differences in order can be difficult to detect and their convolution and pooling operations can obscure the exact pattern positions. CNNs also require all inputs to be of equal dimension, which is usually not the case with sequence data. In the next column, we will introduce alternative network architectures that can address these problems and demonstrate the power of deep learning methods to automatically capture salient features of complex data into low-dimensional representations known as embeddings.

Alexander Derry[1], Martin Krzywinski[2] ✉ & Naomi Altman[3]

[1]Department of Biomedical Data Science, Stanford University, Stanford, CA, USA. [2]Canada's Michael Smith Genome Sciences Centre, Vancouver, British Columbia, Canada. [3]Department of Statistics, The Pennsylvania State University, State College, PA, USA. ✉e-mail: martink@bcgsc.ca

Published online: 14 August 2023

References

1. Derry, A., Krzywinski, M. & Altman, N. *Nat. Methods* **20**, 165–167 (2023).
2. Lever, J., Krzywinski, M. & Altman, N. *Nat. Methods* **13**, 703–704 (2016).
3. Altman, N. & Krzywinski, M. *Nat. Methods* **18**, 840–842 (2021).

Competing interests

The authors declare no competing interests.